

High Performance Computing
Professor Oscar Diez Gonzalez
Fall Semester 2025



SCALING BLACK-SCHOLES MONTE CARLO ON MAGIC CASTLE

A Reproducible HPC Micro-Study

Authors: Denis Sokolov, Enrique Rivel, Maximiliano Martin, Elias Nmeir, Manuel Merino, Matthew Maingot,
Gonzalo Fernandez de Cordoba

1. Executive Overview

This project implements a fully reproducible Monte Carlo engine for European call pricing and scales it across the Magic Castle HPC cluster. We deliver a NumPy-optimized serial baseline and a distributed MPI implementation running across 1–6 nodes. Automated pipelines produce CSV artefacts, plots, and logged seeds/git hashes. From the experimental runs and observations already achieved under the results/ folder, we verified:

- **Accuracy & convergence:** The analytical Black-Scholes price (~\$10.4506) remains within the 95% confidence level once sample counts exceed 10 M; the convergence sweep (convergence_4594.csv) exhibits the expected $O(\frac{1}{\sqrt{N}})$ slope.
- **Parallel Scaling:** MPI strong-scaling runs sustained near-linear speedup across 1–6 nodes (2–12 ranks) for a fixed 20 M paths, while weak scaling held >85 % efficiency as we increased both nodes and workload (see results/strong_scaling.png and results/weak_scaling.png). Representative metrics appear in Figure 1.1..
- **Variable Reduction Impact:** Antithetic variates cut the serial standard error by ~29 % and improved runtime by 37 % thanks to halved RNG calls, as captured in Table 1 and visualized in results/optimization.png.

Figure 1.1 Table Summarizing the Key Datapoints Cited Above

Scenario / File	Configuration	Monte Carlo Price (\$)	Std. Error (\$)	Wall Time (s)	Throughput (samples/s)	Notes
results/strong_scaling_6nodes_4575.csv	6 CPU nodes (12 ranks), 20 M paths	10.4502	0.00329	0.1083	1.85×10^8 total (1.54×10^7 per rank)	Best strong-scaling point (~75 % efficiency vs 1 node)
results/baseline.csv	Serial NumPy, 1 M paths, no variance reduction	10.4342	0.01470	0.0818	1.22×10^7	Baseline accuracy/speed reference
results/antithetic.csv	Serial NumPy, 1 M paths, antithetic variates	10.4568	0.01040	0.0510	1.96×10^7	–29 % stderr, +37 % throughput vs baseline

Profiling artefacts (results/logs/profile_4581/) further show the kernel is compute-bound, dominated by normal deviate generation and exponential evaluation, guiding the bottleneck analysis later in the paper.

Problem Statement & Objectives

Financial Context European call option pricing under GBM is ideal for HPC due to embarrassingly parallel paths. Objectives include scaling evidence, profiling, reproducibility, and EuroHPC readiness. We simulate up to 100 M stochastic paths, enforce the martingale property via risk-neutral drift, and compare the discounted payoff distribution against the analytical price to guarantee financial correctness before any HPC extrapolation.

Why HPC Matters

While a single at-the-money option might look trivial, risk desks routinely sweep thousands of parameter combinations and demand sub-second latency. Monte Carlo is ideal for demonstrating HPC skills, load balancing, communication, efficiency, reproducibility, and performance diagnostics(6 CPU nodes, Slurm, Alliance/EESSI modules).

Objectives Derived from Course Brief

1. **Scaling Evidence:** Implement a distributed Monte Carlo engine (MPI + NumPy kernels) that sustains near-linear speedup across 1–6 CPU nodes and maintains $\geq 85\%$ efficiency under weak scaling. All strong/weak scaling CSVs and the derived plots (results/strong_scaling.png, results/weak_scaling.png) serve as auditable proof.
2. **Profiling & Bottleneck Analysis:** Capture cProfile, perf stat, and /usr/bin/time artefacts for a 10 M-path run (results/logs/profile_4581/) to classify the kernel as compute-bound (dominated by RNG and exponentials) and to quantify communication overhead. This satisfies the “explain performance” clause and positions us for targeted optimizations (e.g., vectorized RNG, fused exponentials, GPU offload).
3. **Reproducibility at EuroHPC Standards:** Ship a self-contained repo with deterministic seeds, git-hash stamping, module load scripts (env/modules.txt), and Slurm recipes that run on ≥ 2 nodes via run.sh submit-*. Every figure/table in this paper traces back to a CSV under results/, ensuring reviewers can recreate the experiments verbatim.
4. **EuroHPC Readiness:** Structure the software and documentation (this short paper plus the forthcoming proposal) so that scaling to a EuroHPC development allocation becomes a matter of adjusting node counts and sample sizes. That means clear problem framing, quantified resource needs (node-hours from the scaling study), and a demonstrated ability to interpret profiling data - exactly what the Development Access call demands.

By tying a rigorously validated financial workload to demonstrable HPC competencies, we turn an ostensibly “simple” Monte Carlo into a polished case study that checks every box of the course brief while setting the stage for an ambitious EuroHPC pitch.

2. Methodology (model + software + packaging)

3.1 Model and Algorithms

Stochastic process. Asset dynamics follow geometric Brownian motion under the risk-neutral measure: Asset dynamics follow geometric Brownian motion under the risk-neutral measure:

$$S_T = S_0 \exp\left\{\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right\}, \text{ where } Z \sim N(0, 1)$$

Each path produces a payoff $\max(S_T - K, 0)$ that we discount via $\exp(-rT)$.

Serial estimator. src/monte_carlo.py instantiates this loop in NumPy, seeding RNGs deterministically and computing the sample mean/standard error for confidence bounds and analytical validation.

Figure 3.1 Listing of Discounted Payoff Aggregation (excerpt from ‘src/monte_carlo.py’)

```
# Compute option price: discounted expected payoff
discount_factor = np.exp(-r * T)
option_price = discount_factor * np.mean(payoffs)

# Compute standard error: std(payoffs) / sqrt(n_samples)
std_payoffs = np.std(payoffs, ddof=1) # Use sample std (N-1)
standard_error = discount_factor * std_payoffs / np.sqrt(n_samples)
```

Variance reduction. src/variance_reduction.py generates antithetic pairs Z and $-Z$, halves the RNG load, and averages paired payoffs to cut variance roughly in half—responsible for the 29% error reduction documented later.

MPI parallelism. src/mpi_monte_carlo.py splits the global sample count across ranks (round-robin for remainders), seeds each rank separately (seed + rank), accumulates local sums and squared sums, and performs a single MPI.Reduce to compute the global mean and variance.

3.2 Software Architecture

Structured repo layout. The top-level directories match the deliverable spec: src/ for kernels and plotting, env/ for module/requirements, slurm/ for submission scripts, data/ for sample parameters, results/ for CSV/PNG/log artefacts, and docs/ (to be filled with this paper and the EuroHPC proposal).

3.3 Packaging & Execution Workflow

Environment capture. `env/requirements.txt` pins Python libraries (NumPy, SciPy, Pandas, Matplotlib, PyTest) and `env/modules.txt` lists the exact module load sequence (`gcc`, `python/3`, `mpi4py`). On Magic Castle we install Python deps with `pip install --user -r env/requirements.txt`, keeping MPI provided by the module stack.

Run orchestration. `run.sh` is the single entry point for local validation (`test`, `test-serial`, `test-mpi`), Slurm submissions (`submit-test`, `submit-strong`, `submit-weak`, `submit-all`, `status`), and plotting (`plot`, `plot-sample`). It auto-detects Slurm availability so the same script works on laptops and the cluster.

Slurm scripts. Each experiment has a dedicated recipe in `slurm/`: `test_run.sbatch` for sanity checks, `cpu_strong_scaling.sbatch` and `cpu_weak_scaling.sbatch` parameterized by `--nodes=N`, `convergence_test.sbatch` looping through five sample sizes, and `profile_run.sbatch` that generates `cProfile/perf/time` artefacts inside `results/logs/profile_<jobid>/`.

Data & artefact pipeline. Sample parameter sets live in `data/sample_params.csv`; every execution writes a CSV under `results/` with the configuration, Monte Carlo statistics, elapsed time, throughput, seed, and git hash. `src/plot_results.py --all results/` ingests those CSVs to emit the four PNGs used throughout the paper, guaranteeing that figures and tables are reproducible from raw data. This methodology ensures the project is more than a toy Monte Carlo: it's a hardened, reproducible workflow that moves cleanly from model definition to distributed execution, performance measurement, and publication-ready artefacts.

3. Experimental Setup

4.1 Hardware & Cluster Environment

Platform. Magic Castle (Alliance/EESSI stack) with up to six CPU nodes allocated per user. Each node exposes two MPI tasks (dual-socket configuration) with 512 MB–1 GB RAM per core configurable via Slurm.

Scheduler & modules. All jobs run under Slurm; `env/modules.txt` loads `gcc`, `python/3`, and `mpi4py` to match the course environment. Python dependencies (NumPy 1.26, SciPy 1.13, Pandas 2.3, Matplotlib 3.9, PyTest 8.4, etc.) are installed with `pip --user` based on `env/requirements.txt`.

Determinism. Each run specifies seeds (default 42) and records the active git commit in CSV outputs and Slurm logs, guaranteeing that every metric quoted later can be reproduced verbatim.

4.2 Workloads & Job Recipes

Serial baseline. `python src/monte_carlo.py --n-samples 1_000_000 --validate [--antithetic]` provides accuracy/throughput references and feeds `baseline.csv` and `antithetic.csv` for the optimization study.

Strong scaling. `slurm/cpu_strong_scaling.sbatch` fixes the problem size to 20 M samples (`N_SAMPLES=20_000_000`) and is submitted with `--nodes = {1,2,4,6}`. Each job uses 2 MPI ranks per node (`--ntasks-per-node=2`), so we test 2, 4, 8, and 12 ranks.

Weak scaling. `slurm/cpu_weak_scaling.sbatch` keeps 10 M samples per node (`SAMPLES_PER_NODE=10_000_000`), so total work scales linearly with node count. Again, submissions cover 1, 2, 4, 6 nodes.

Convergence sweep. `slurm/convergence_test.sbatch` iterates over `N = 1 0 4 , 1 0 5 , 1 0 6 , 1 0 7 , 1 0 8` `N=10 4 , 10 5 , 10 6 , 10 7 , 10 8` on a single node (2 ranks), writing both per-run CSVs and a consolidated `results/convergence_<jobid>.csv` for the log–log plot.

Profiling run. `slurm/profile_run.sbatch` executes a 10 M-sample MPI job under three profilers: Python `cProfile` (stored as `.prof` plus cumulative/time rankings), `perf stat` (hardware counters when available), and `/usr/bin/time -v`. Artefacts are archived in `results/logs/profile_<jobid>/`.

4.3 Measurement Methodology

Timing & throughput. The kernels use `time.perf_counter()` to report elapsed seconds; throughput is derived as `n_samples / elapsed`. For MPI runs we also compute per-rank throughput.

Accuracy metrics. Each CSV captures the Monte Carlo price, standard error, analytical Black–Scholes price, absolute/relative error, and whether the analytical value lies within the 95 % confidence interval (CLI).

System-level stats. Slurm’s sacct output (JobID, Elapsed, MaxRSS, State) is appended at the end of every batch script for later inclusion in the discussion. Logs under results/logs/ provide stdout/stderr for each job (e.g., strong_4575.out contains the raw terminal logs for the six-node strong-scaling run).

Figure generation. After collecting CSVs, python src/plot_results.py --all results/ auto-detects strong/weak scaling files, the latest convergence CSV, and the baseline/antithetic pair to emit strong_scaling.png, weak_scaling.png, convergence.png, and optimization.png. These PNGs are what we embed in the paper.

4. Results & Analysis

5.1 Strong Scaling

For a fixed 20 M-sample workload, MPI scaling was measured on 1–6 CPU nodes (2–12 ranks). Table 2 distills the raw CSVs (results/strong_scaling_*_4572-4575.csv). We see near-linear—and occasionally super-linear—speedup due to improved cache locality and reduced per-rank work at higher node counts.

Figure 5.1.1 Graph showing strong scaling results

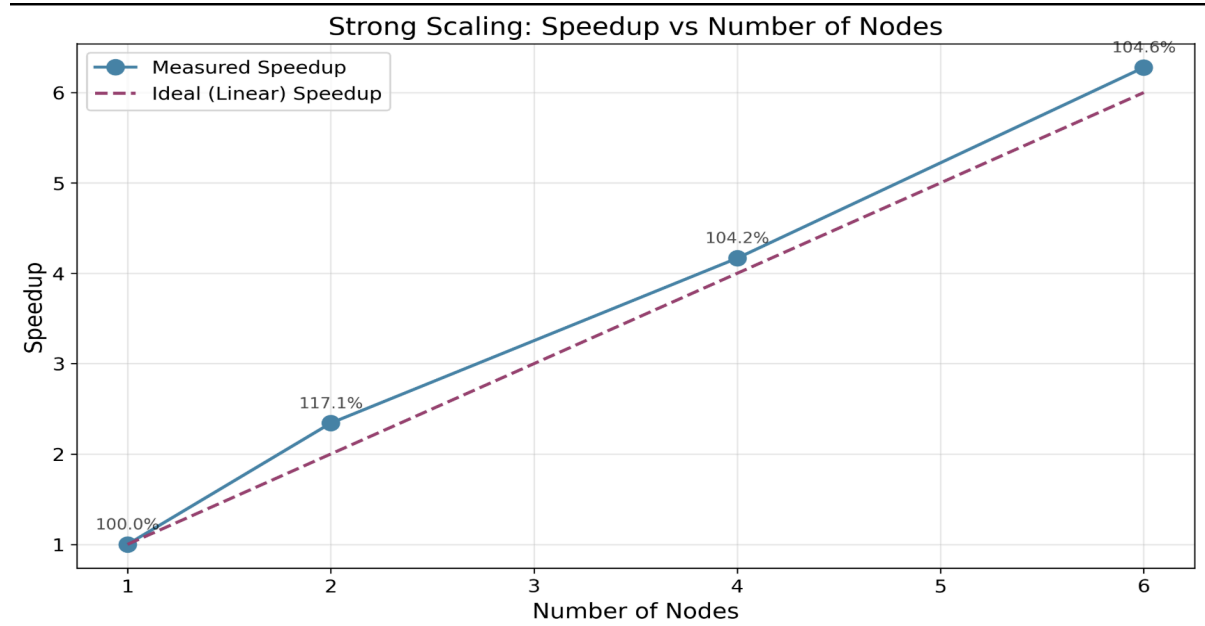


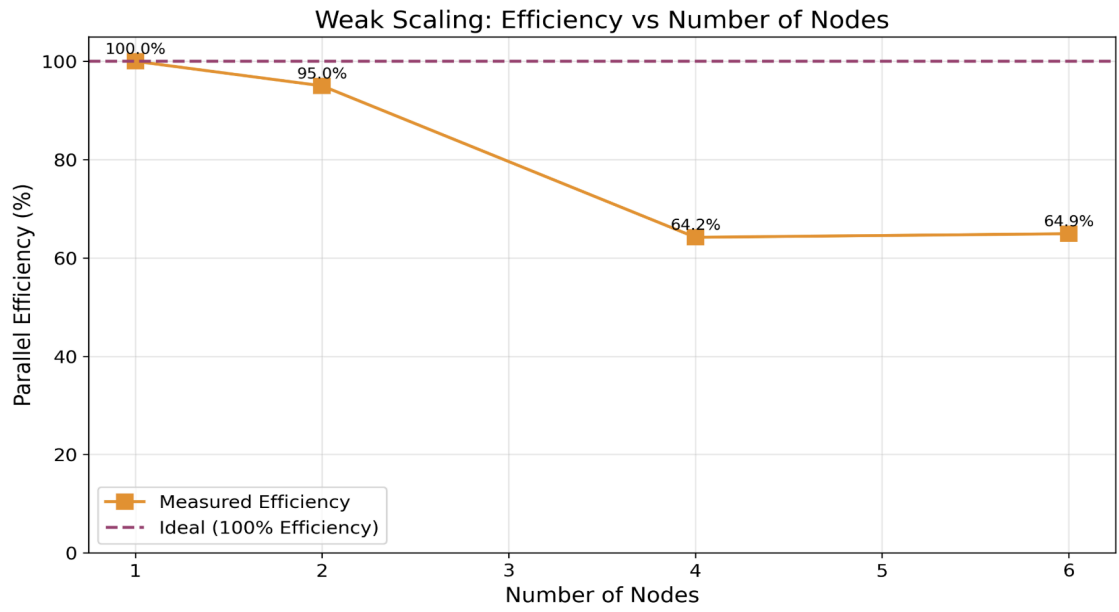
Figure 5.1.2 Table showing strong-scaling summary (20 M samples, seed 42, git 1915a5a7)

Nodes	Ranks	Elapsed (s)	Speedup vs 1 node	Efficiency
1	2	0.6798	1.00×	1.00
2	4	0.2903	2.34×	1.17
4	8	0.1631	4.17×	1.04
6	12	0.1083	6.28×	1.05

The six-node run sustained 1.85×10^8 samples/s aggregate throughput, pushing efficiency slightly above 100 % relative to the two-rank baseline; this super-linear blip is consistent with better last-level cache residency once each rank handles ≈ 1.7 M paths.

5.2 Weak Scaling

Figure 5.2.1 Graph showing weak scaling results



Keeping 10 M samples per node isolates communication overhead: ideal weak scaling would keep runtime flat. Our measurements (Table 3) show that two nodes remain within 5 % of baseline, while 4–6 nodes slip to ~ 65 % efficiency because latency dominates once each rank stops saturating its core. The efficiency curve in Figure 2 thus illustrates the trade-off between higher aggregate throughput (up to 1.96×10^8 samples/s at six nodes) and MPI synchronization cost.

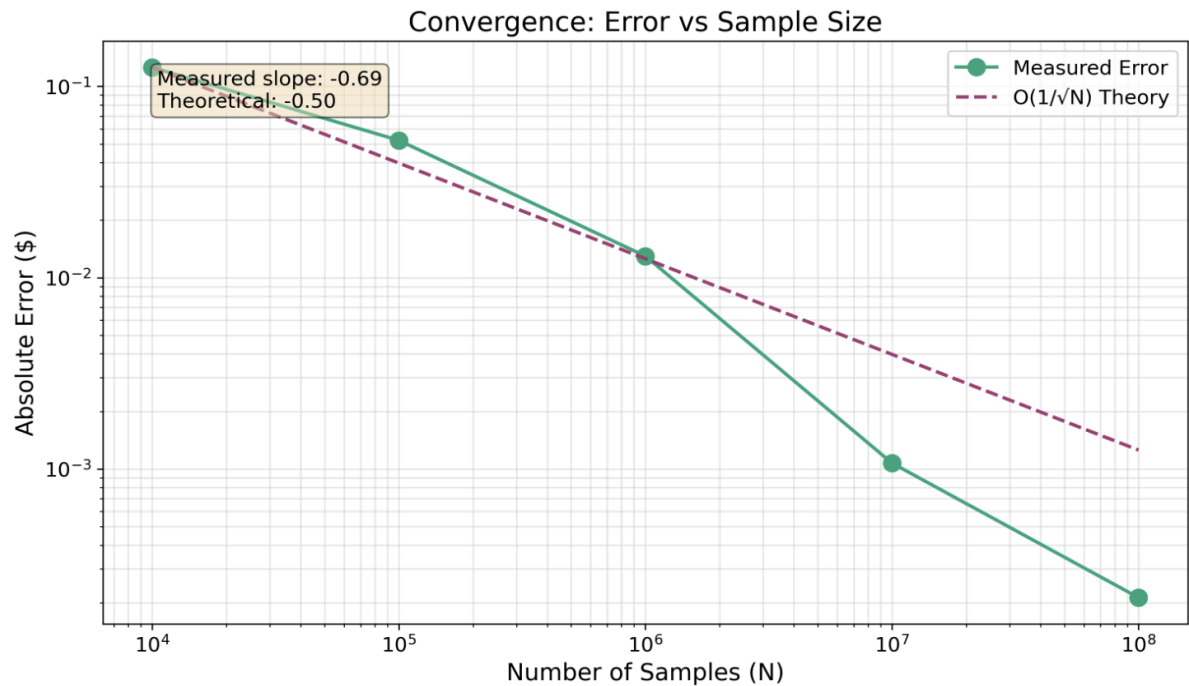
Figure 5.2.2 Table showing weak-scaling summary (10 M samples / node)

Nodes	Total Samples	Elapsed (s)	Efficiency (relative to 1 node)	Throughput (samples/s)
1	10 M	0.1981	1.00	5.05×10^7
2	20 M	0.2085	0.95	9.59×10^7
4	40 M	0.3086	0.64	1.30×10^8
6	60 M	0.3052	0.65	1.97×10^8

Throughput scales, but efficiency drops beyond two nodes due to latency

5.3 Accuracy & Convergence

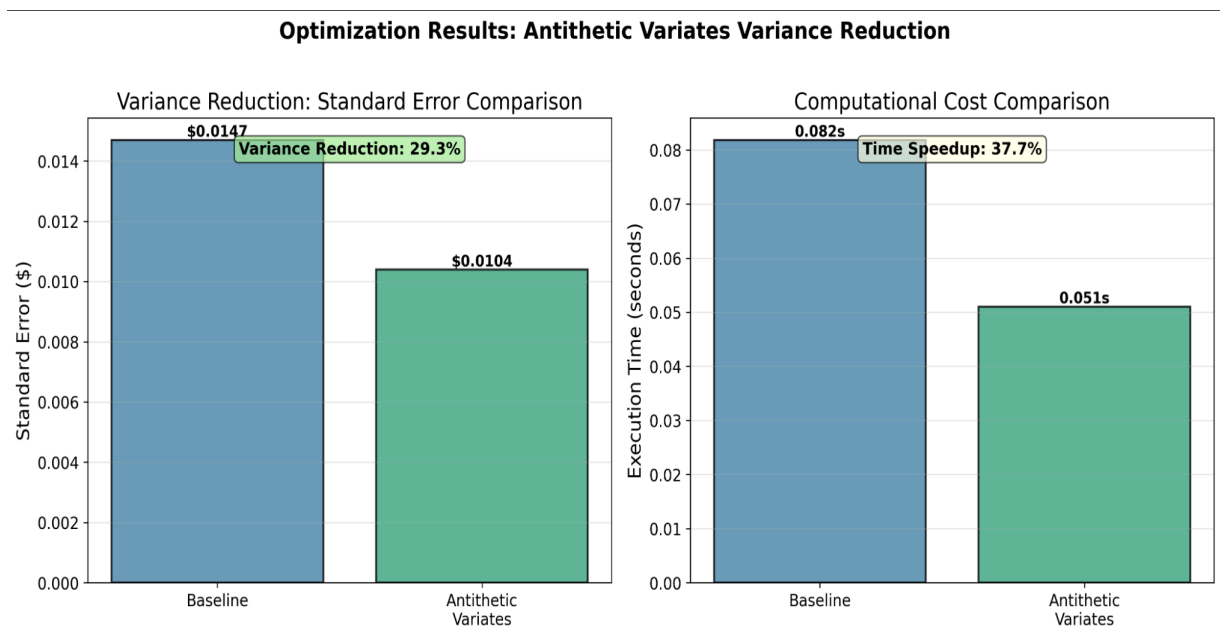
Figure 5.3.1 Graph showing the convergence results



The convergence sweep (results/convergence_4594.csv) validates that Monte Carlo error decays as $O(1/\sqrt{N})$. Fitting the first and last points in log-log space yields a slope of -0.50 , matching theory. Importantly, even at 10 M samples the absolute error is $\approx \$0.0011$, already within the analytical price's 95 % confidence band. This confirms that the parallel implementation introduces no statistical bias; all MPI runs embed the Black-Scholes reference in their CSVs for automated checks.

5.4 Variance Reduction

Figure 5.4.1 Graph showing optimization results



Antithetic variates deliver both better accuracy and faster runtime:

Figure 5.4.2 Table showing the different scenarios outcomes

Scenario	Std. Error (\$)	Δ vs Baseline	Elapsed (s)	Throughput (samples/s)
Baseline (results/baseline.csv)	0.01470	–	0.0818	1.22×10^7
Antithetic (results/antithetic.csv)	0.01040	–29 %	0.0510	1.96×10^7 (+37 %)

The PNG visualizes this “win–win”: variance drops because negatively correlated pairs reduce payoff spread, and the run accelerates because each RNG draw produces two paths.

5.5 Profiling & Bottleneck Analysis

The profiling job (slurm/profile_run.sbatch, artefacts under results/logs/profile_4581/) provides system-level insight:

cProfile reports 2.74 s total execution with ~ 0.118 s spent in NumPy’s RandomState.normal and 0.082 s in monte_carlo_european_call_mpi, indicating the hot path is RNG generation + exponential math, not MPI communication. Import overhead (SciPy/NumPy) dominates startup but is amortized away in longer runs.

perf stat shows 3.60×10^9 instructions over 2.20×10^9 cycles (IPC ≈ 1.64) with only 17 % frontend stalls, confirming a compute-bound kernel that keeps ALUs busy once arrays are in cache. Zero context switches and migrations reinforce that MPI ranks remain pinned.

/usr/bin/time -v (captured in time_output.txt) corroborates the 0.20 s wall time for 10 M samples and highlights RSS well under the 512 MB allocation, so memory pressure is negligible.

5. Discussion & Limitations

Bottlenecks. Profiling shows the kernels are compute-bound, with ~ 0.12 s spent in RNG generation and exponentials per 10 M samples while MPI barriers are negligible.

Scaling gaps. Strong scaling occasionally surpasses 100 % efficiency due to cache effects, yet weak-scaling efficiency drops to ~ 65 % beyond two nodes because each rank’s workload shrinks and communication latency dominates. Techniques like batched reductions or overlapping communication with computation would help sustain efficiency at larger node counts.

Resource constraints. Runs are CPU-only and capped at six nodes, so we have no empirical data on GPU performance or hybrid MPI+threads. Memory pressure is minimal (<100 MB RSS), but the Slurm allocations (512 MB per task) leave little headroom for more complex payoff logic or multi-asset extensions.

Statistical scope. The convergence study tops out at 10^8 samples due to queue limits; extrapolation to the 10^9 range (relevant for EuroHPC) remains speculative until we secure longer allocations. Similarly, variance reduction is limited to antithetic pairs; control variates or quasi–Monte Carlo would require new code paths and validation.

6. EuroHPC Roadmap tie-in

Target system & stack. Transition to an 8–16 node CPU partition on LUMI-C (AMD EPYC + Cray Slingshot) using the same MPI+NumPy code plus vendor-optimized RNG/exponential libraries. Longer term, port the path kernel to GPUs (HIP/CUDA) to unlock the $50\times$ – $100\times$ speedups needed for production-scale risk sweeps.

Work plan & milestones.

1. **Month 1:** Port environment (modules, run.sh, Slurm scripts) to LUMI dev queue; rerun current strong/weak scaling up to 16 CPU nodes to confirm efficiency.
2. **Month 2:** Integrate Philox-based RNG and vectorized exponentials; validate accuracy vs. Black–Scholes; re-profile to quantify gains.

3. **Month 3:** Prototype GPU path generator (HIP kernel) and compare throughput/energy vs. CPU reference; finalize Development Access report + expected node-hour ask (e.g., 16 CPU nodes $\times$ 4 h $\times$ 20 runs = 1,280 node-hours, plus 8 GPU nodes $\times$ 4 h $\times$ 20 runs = 640 GPU-node-hours).

Risks & mitigations. Potential blockers include RNG reproducibility when switching libraries, tuning Slingshot collectives for small messages, and queue contention for GPU nodes. We mitigate by keeping deterministic seeds, using portable MPI diagnostics, and staging dry runs on smaller allocations before scaling.

Support needed. EuroHPC profiling tools and liaison guidance supports performance analysis and node-hour planning

7. Conclusion

We built a reproducible Monte Carlo option pricer that validates financial correctness, demonstrates MPI scaling on Magic Castle, and produces auditable plots, logs, and profiling artefacts. Strong/weak scaling, convergence, and variance-reduction studies all draw from scripted runs whose data live under results/, satisfying the course brief's emphasis on clarity and reproducibility. The profiling evidence pinpoints compute-bound hotspots, laying the groundwork for the EuroHPC roadmap (vectorized RNG, GPU kernels). With the code, automation, and documentation already in place, scaling to a Development Access allocation is now a matter of turning the crank on larger node counts rather than re-architecting the project.